

一.学习内容引入

1.相关岗位

嵌入式应用开发工程师：必备的知识

- 1、电子信息、通信、自动化等相关专业，统招本科以上学历，5年以上嵌入式软件开发工作经验。
- 2、熟练掌握 C/C++ 语言编程，有丰富的Linux/Android 嵌入式软件开发调试经验。
- 3、熟悉Linux内核机制，熟练掌握Linux下应用软件开发技能，具备多进程/线程、网络/IO/摄像头等编程经验，熟悉UART、SPI、I2C、CAN、USB等各类外设通信协议。
- 2、具备良好的C和C++编程能力和调试技巧，具备良好的编程素养，具有良好的数据结构和算法基础；
- 3、具备良好的操作系统知识，熟悉多线程开发；
- 4、具有在Windows或Linux上编写应用程序的经验，熟悉ARM体系结构，具有编写、调试ARM上Linux应用程序的经验（Cortex-A9/A7及以上平台）；

2.学习的主要内容

多进程

进程相关的概念

linux提供进程操作有关的接口函数

进程间的通信

多线程

线程有关的概念

linux提供的线程有关的接口函数

线程的同步与互斥

3.可以解决的实际问题

系统编程为了实现多任务同时执行(多任务的并发操作)

例子1：360/电脑管家--》可以同时干多件事情(杀毒+清除垃圾+扫描漏洞)

例子2：西游记孙悟空大闹天宫--》拔猴毛，变出小猴子--》实现同时打十万天兵天将

二.多进程

1.相关的概念

进程：一个正在运行的程序，只要你运行了程序，那么系统中都会产生一个对应的进程，动态的概念

程序：用编译器编译得到的二进制文件，静态的概念

相关的shell命令： ps -elf 查看当前系统进程的状态信息

备注：ps英文全称process state

top 动态查看进程的运行状态信息，输入q退出

kill -9 进程ID号 //给指定的进程发送9号信号

kill -SIGINT 进程的ID号

kill -l //罗列linux系统中所有的信号

```
gec@ubuntu:/mnt/hgfs/share$ kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

比如：SIGINT是2号信号，作用就是终止当前进程、我们平时ctrl+c强行退出程序，ctrl+c系统会自动给当前进程发送SIGINT信号

SIGKILL是9号信号，作用就是杀死、终止当前进程

killall -9 进程的名字 //给指定的进程发送9号信号

进程的组成：代码块(程序代码)+数据块(程序中用到的数据)+进程控制块

进程控制块(PCB-->process contrl block)：指的是系统中定义的一个结构体,当你./程序名 运行一个程序的时候，系统里面产生对应名字的进程，并且把该进程在运行时候的状态参数全部存放到这个类型的结构体变量中

系统相关的头文件/usr/src/linux-headers-4.10.0-28/include/linux/sched.h这个头文件1512行定义了进程控制块有关的结构体

```
struct task_struct //专业术语叫做进程控制块
{
    long state //保存进程的运行状态
        -1表示没有运行
        0表示正在运行
        >0表示暂停
    int pid; //进程的ID号
    long prriority //进程的优先级
}
```

父进程：哪个主函数调用fork创建子进程，该程序就是父进程

孤儿进程：父进程优先于子进程退出了，导致子进程的资源被系统中init进程(进程ID是1)回收，这种现象就叫做孤儿进程

僵尸进程：子进程退出了,但是父进程没有主动调用wait/waitpid函数去回收该子进程,此时这个子进程就是僵尸进程

2.linux提供跟进程有关的接口函数

(1)进程的创建(创建子进程)

```
pid_t fork();
```

返回值： >0 表示父进程,此时返回值中存放的是子进程的id号

==0 表示子进程

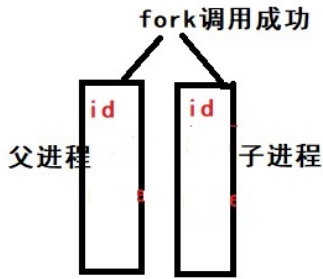
-1 表示失败

fork函数的特点

第一个： 一次调用，两次返回，分别表示父子进程

前面学习的函数，一次调用，只能返回一次

fork的底层原理分析



//父进程的代码

pid_t id;

id = fork();

if (id > 0) 用父进程的局部变量id来判断的

{

//父进程的代码

}

else if (id == 0) 用子进程拷贝过来的相同名字的局部变量id来判断的

{

//子进程的代码

}

//父进程的代码

1. 操作系统会给予子进程单独申请额外的系统资源(cpu的使用权, 内存), 让子进程可以独立运行

2. 子进程会把父进程的变量拷贝一份到子进程自己的内存空间

第二个: fork创建子进程的时候, 会给予子进程分配独立的内存空间, 同时子进程会复制父进程的所有资源(子进程会复制父进程调用fork之前的变量(全局变量和局部变量)和代码)

(2)进程的退出

void exit(int status);

return和exit的区别:

区别一: return 是关键字, exit()是函数

区别二: return结束函数, 返回返回值给调用者. exit()用于结束整个进程

(3)进程的回收

函数1: pid_t wait(int *stat_loc);

返回值: 成功 回收到的那个子进程的id号

失败 -1

参数(重点): stat_loc --》存放进程退出时候的状态信息

进程的退出值(exit/_exit圆括号里面的整型参数)仅仅只是状态信息的一部分, 状态信息还包含其它内容(子进程是正常退出还是异常退出, 是被哪个信号弄死的)

特点: 阻塞父进程, 等待子进程的退出

超纲拓展一下: stat_loc是如何存放那么多状态信息的?

bit0 ~ bit6	7 bit	终止信号编号 (子进程被信号杀死时有效)
bit7	1 bit	Core Dump 标志位: 1 = 产生 core 转储文件, 0 = 未生成
bit8 ~ bit15	8 bit	进程正常退出时的返回码 (exit(status) / return 的低8位)
bit16 ~ bit31	16 bit	未使用, 固定为 0

函数2: pid_t waitpid(pid_t pid, int *stat_loc, int options);

返回值: 成功 回收到的那个子进程的id号

失败 -1

参数: pid --》有如下四种情况

< -1 回收进程组id是pid绝对值中的某个进程

== -1 回收任意一个进程
== 0 回收本进程组中的某个进程
> 0 指定回收进程id是pid的这个进程(最常用)
stat_loc --》跟wait的参数意思一模一样
options --》 设置为0表示阻塞父进程,让父进程等待子进程退出之后回收子进程

宏	含义
WIFEXITED(status) ^①	如果子进程正常退出，则该宏为真。
WEXITSTATUS(status)	如果子进程正常退出，则该宏将获取子进程的退出值。
WIFSIGNALED(status)	如果子进程被信号杀死，则该宏为真。
WTERMSIG(status)	如果子进程被信号杀死，则该宏将获取导致他死亡的信号值。
WCOREDUMP(status) ^②	如果子进程被信号杀死且生成核心转储文件（core dump），则该宏为真。
WIFSTOPPED(status)	如果子进程的被信号暂停，且 option 中 WUNTRACED 已经被设置时，则该宏为真。
WSTOPSIG(status)	如果 WIFSTOPPED(status)为真，则该宏将获取导致子进程暂停的信号值。
WIFCONTINUED(status)	如果子进程被信号 SIGCONT 重新置为就绪态，该宏为真。

表 5-5 处理子进程退出状态值的宏

(4)获取进程的id以及获取父进程的id

```
获取当前进程的id号
pid_t getpid(void);

获取父进程的id
pid_t getppid(void);
```

三.练习作业

- 1.创建两个子进程，父进程实现杀毒(模拟),第一个子进程清除垃圾，第二个子进程扫描漏洞
- 2.用进程相关API函数编写一个程序，使之产生一个进程链：父进程派生一个子进程后,第一个子进程打印出自己的PID，然后退出，父进程继续派生子进程，子进程打印PID，然后退出，以此类推。总共创建三个子进程，要求父进程回收所有的子进程(按照指定的顺序回收：先回收子进程2，再回收子进程3，再回收子进程1)

创建的时候是1 2 3 回收的时候 2 3 1

- 3.创建两个子进程，一个子进程循环每隔1秒显示一张bmp图片

另外一个子进程帮你播放歌曲

背景知识开发板：有个音频播放器叫做madplay

播放音乐：secure CRT直接输入 madplay 音频文件的绝对路径

madplay /1.mp3

代码播放音乐：

```
#include <stdlib.h>

int system(const char *command);
```

参数：command --》你要执行的某个指令

```
system("madplay /1.mp3");
```